

GVFG 客戶 API Reference

目錄

1. GVFG API 結構與常數

- 1.1 公開常數
- 1.2 gvfg_status_t
- 1.3 gvfg_pixel_format_t
- 1.4 gvfg_channel_t
- 1.5 gvfg_video_interface_t
- 1.6 gvfg_sdi_mode_t
- 1.7 gvfg_sdi_resolution_t
- 1.8 gvfg_sdi_fps_t
- 1.9 gvfg_device_info_t
- 1.10 gvfg_sdi_info_t
- 1.11 gvfg_signal_status_t
- 1.12 gvfg_runtime_info_t
- 1.13 gvfg_frame_t
- 1.14 gvfg_audio_format_t
- 1.15 gvfg_audio_frame_t
- 1.16 gvfg_gpu_output_format_t
- 1.17 gvfg_gpu_output_buffer_t
- 1.18 gvfg_event_type_t
- 1.19 gvfg_handle

2. GVFG API 函式

- 2.1 gvfg_enumerate_devices
- 2.2 gvfg_create
- 2.3 gvfg_destroy
- 2.4 gvfg_open_channel

2.5 gvfg_get_channel_sdi_info

- 2.6 Zero-copy mode selection
- 2.7 gvfg_set_channel_video_format
- 2.8 Audio capture selection and frame ownership
- 2.9 gvfg_start_channel
- 2.10 gvfg_read_channel_frame
- 2.11 gvfg_release_channel_frame
- 2.12 gvfg_gpu_convert_to_buffer
- 2.13 gvfg_gpu_convert_to_bgra8
- 2.14 gvfg_gpu_convert_to_rgb10a2
- 2.15 gvfg_gpu_convert_to_nv12
- 2.16 gvfg_poll_channel_event
- 2.17 gvfg_stop
- 2.18 gvfg_stop_channel
- 2.19 gvfg_get_channel_signal_status
- 2.20 gvfg_get_channel_runtime_info
- 2.21 gvfg_get_version
- 2.22 gvfg_pixel_format_name
- 2.23 gvfg_strerror
- 2.24 gvfg_get_channel_last_sdk_error_detail

3. Preview API 完整參考

- 3.1 Preview 型別
- 3.2 Preview 函式

4. Audio Playback Helper 完整參考

- 4.1 Audio Playback 型別
- 4.2 Audio Playback 函式

1. GVFG API 結構與常數

說明 `gvfg_capture.h` 的所有公開常數、型別、結構。除非特別註明，所有輸出結構都建議先用 `{0}` 初始化。

1.1 公開常數

常數	值	說明
<code>GVFG_MAX_DEVICES</code>	16	一次最多列舉的裝置數
<code>GVFG_TIMEOUT_INFINITE</code>	<code>UINT32_MAX</code>	無限等待； <code>timeout_ms == 0</code> 則完全不等待

1.2 `gvfg_status_t`

`gvfg_status_t` 是 32-bit 整數。0 表示成功，失敗時回傳負數的 `GVFG_E*` 狀態。

成員	值	說明
<code>GVFG_OK</code>	0	成功
<code>GVFG_EINVAL</code>	-1	NULL pointer、無效 channel、buffer 大小或 token 等參數錯誤
<code>GVFG_ENODEV</code>	-2	找不到裝置或裝置無法開啟
<code>GVFG_ESTATE</code>	-3	呼叫順序或目前 session 狀態不允許此操作
<code>GVFG_EIO</code>	-4	SDK I/O、GPU 或其他處理失敗
<code>GVFG_ENOTSUP</code>	-5	不支援指定功能或格式
<code>GVFG_ETIMEOUT</code>	-6	在期限內等不到 frame、event 或操作完成
<code>GVFG_EBUSY</code>	-7	裝置或 capture resource 正在使用中

統一以 `status != GVFG_OK` 判斷失敗，不可使用 Windows `FAILED()` / `SUCCEEDED()`。 `gvfg_strerror()` 回傳對應的公開英文錯誤說明。

1.3 `gvfg_pixel_format_t`

成員	值	說明
<code>GVFG_PIXFMT_UNKNOWN</code>	0	未知或尚無有效訊號格式
<code>GVFG_PIXFMT_YUY2</code>	1	8-bit packed YUV 4:2:2；byte order Y0 U0 Y1 V0，每 pixel 2 bytes
<code>GVFG_PIXFMT_Y210</code>	2	10-bit packed YUV 4:2:2；目前每 pixel 4 bytes

1.4 `gvfg_channel_t`

成員	值	說明
<code>GVFG_CHANNEL_0</code>	0	裝置輸入通道 0
<code>GVFG_CHANNEL_1</code>	1	裝置輸入通道 1

1.5 gvfg_video_interface_t

成員	值	說明
GVFG_INPUT_INTERFACE_SDI	0	SDI input
GVFG_INPUT_INTERFACE_HDMI	1	HDMI input

1.6 gvfg_sdi_mode_t

成員	值	說明
GVFG_SDI_INPUT_MODE_HD	0	HD-SDI input
GVFG_SDI_INPUT_MODE_SD	1	SD-SDI input
GVFG_SDI_INPUT_MODE_3G	2	3G-SDI input

1.7 gvfg_sdi_resolution_t

成員	值	說明
GVFG_SDI_INPUT_RESOLUTION_SMPTE_ST_274_1920X1080	0x0	SMPTE ST 274 · 1920 x 1080
GVFG_SDI_INPUT_RESOLUTION_SMPTE_ST_296_1280X720	0x1	SMPTE ST 296 · 1280 x 720
GVFG_SDI_INPUT_RESOLUTION_SMPTE_2048_2048X1080	0x2	SMPTE 2048 · 2048 x 1080
GVFG_SDI_INPUT_RESOLUTION_SMPTE_295_1920X1080	0x3	SMPTE 295 · 1920 x 1080
GVFG_SDI_INPUT_RESOLUTION_NTSC_720X486	0x8	NTSC · 720 x 486
GVFG_SDI_INPUT_RESOLUTION_PAL_720X576	0x9	PAL · 720 x 576
GVFG_SDI_INPUT_RESOLUTION_UNKNOWN	0xF	未知或不支援的 resolution

1.8 gvfg_sdi_fps_t

成員	值	說明
GVFG_SDI_INPUT_FPS_NONE	0x0	無可用 frame rate
GVFG_SDI_INPUT_FPS_23_98	0x2	23.98 fps
GVFG_SDI_INPUT_FPS_24	0x3	24 fps
GVFG_SDI_INPUT_FPS_47_95	0x4	47.95 fps
GVFG_SDI_INPUT_FPS_25	0x5	25 fps
GVFG_SDI_INPUT_FPS_29_97	0x6	29.97 fps
GVFG_SDI_INPUT_FPS_30	0x7	30 fps
GVFG_SDI_INPUT_FPS_48	0x8	48 fps
GVFG_SDI_INPUT_FPS_50	0x9	50 fps
GVFG_SDI_INPUT_FPS_59_94	0xA	59.94 fps

成員	值	說明
GVFG_SDI_INPUT_FPS_60	0xB	60 fps

1.9 gvfg_device_info_t

由 `gvfg_enumerate_devices()` 填入。

欄位	型別	說明
name	char[128]	供 UI / log 顯示的 UTF-8、null-terminated 裝置名稱

1.10 gvfg_sdi_info_t

由 `gvfg_get_channel_sdi_info()` 填入。數值欄位可供程式判斷；對應的 `*_name` 欄位是可直接用於 UI 或 log 的 null-terminated 英文字串。

欄位	型別	說明
connected	int	非 0 表示 SDI 輸入訊號已 lock
mode	gvfg_sdi_mode_t	偵測到的 SDI link mode
resolution	gvfg_sdi_resolution_t	偵測到的 SDI resolution code
fps	gvfg_sdi_fps_t	偵測到的 SDI frame-rate code
progressive	int	非 0 為 progressive；0 為 interlaced
level_b	int	非 0 表示 3G-SDI Level B
st352_payload	uint32_t	SMPTE ST 352 payload identifier
error_count	uint32_t	裝置回報的 signal error count
signal_lock_name[32]	char[32]	Signal lock 說明
mode_name[16]	char[16]	SDI mode 名稱
resolution_name[64]	char[64]	Resolution 名稱
fps_name[24]	char[24]	Frame-rate 名稱
scan_name[16]	char[16]	Scan mode 名稱
st352_format_name[64]	char[64]	ST 352 format 名稱
st352_fps_name[24]	char[24]	ST 352 frame-rate 名稱
st352_chroma_name[36]	char[36]	ST 352 chroma 名稱
st352_bit_depth_name[24]	char[24]	ST 352 bit-depth 名稱

1.11 gvfg_signal_status_t

由 `gvfg_get_channel_signal_status()` 填入。

欄位	型別	說明
connected	int	非 0 表示查詢的 channel 目前有有效輸入訊號
channel	int	本次查詢對應的 <code>gvfg_channel_t</code> 值
width	int	連線時的輸入寬度；未連線時為 0
height	int	連線時的輸入高度；未連線時為 0
pixel_format	int	實際 frame payload 的 <code>gvfg_pixel_format_t</code> 值
bit_depth	int	從 payload 格式取得的每色彩 channel bit depth
video_interface	int	<code>gvfg_video_interface_t</code> ，為 SDI 或 HDMI

1.12 `gvfg_runtime_info_t`

由 `gvfg_get_channel_runtime_info()` 填入。Channel 從 stopped 成功進入新的 running session 時重設統計；對已 running 的 channel 再次呼叫 Start 不會重設。

欄位	型別	說明
capture_fps	double	依 <code>gvfg_read_channel_frame()</code> 成功交付時間估算的 FPS；尚無足夠 frame 時為 0
delivered_frames	uint64_t	此次 running session 中成功交付給 caller 的 frame 數

1.13 `gvfg_frame_t`

由 `gvfg_read_channel_frame()` 填入，也是稍後傳給 `gvfg_release_channel_frame()` 的 frame descriptor。

欄位	型別	說明
data	const void *	借用的 frame buffer；release 或 stop 後失效
data_size	uint64_t	此 frame payload 的總 byte 數
width	int	frame 寬度，單位 pixel
height	int	frame 高度，單位 pixel
row_stride_bytes	int	相鄰兩列起點間距，單位 byte；處理每列時必須使用此值
pixel_format	int	<code>gvfg_pixel_format_t</code> 值
bit_depth	int	原生 frame 每色彩 channel 的 bit depth
frame_id	uint64_t	SDK 成功交付序號；每次 channel Start 從 1 重新開始
timestamp_ns	uint64_t	SDK 交付時間；與 audio 共用 monotonic clock，單位 ns

Caller 應把 read 取得的 descriptor 傳回 release；SDK 以 `data + frame_id` 確認目前 held frame。

1.14 `gvfg_audio_format_t`

由 `gvfg_get_channel_audio_format()` 填入，只包含 Application 建立播放或錄音 格式所需的 PCM metadata。

欄位	型別	說明
sample_rate	uint32_t	每秒 sample 數
channels	uint32_t	interleaved PCM channel 數
bits_per_sample	uint32_t	每個 PCM sample 的 bit 數

每個 audio frame 的有效資料長度由 `gvfg_audio_frame_t.data_size` 提供。

1.15 gvfg_audio_frame_t

由 `gvfg_read_channel_audio_frame()` 填入，也是稍後傳給 `gvfg_release_channel_audio_frame()` 的 audio frame descriptor。

欄位	型別	說明
data	const void *	借用的 interleaved PCM；release 或 stop 後失效
data_size	uint64_t	此 frame 的有效 PCM byte 數
sample_rate	uint32_t	每秒 sample 數
channels	uint32_t	interleaved PCM channel 數
bits_per_sample	uint32_t	每個 PCM sample 的 bit 數
frame_id	uint64_t	SDK 成功交付序號；每次 channel Start 從 1 重新開始
timestamp_ns	uint64_t	SDK 交付時間；與 video 共用 monotonic clock，單位 ns

與 video 相同，每個 channel 同時只能持有一個 audio frame；release 以 `data + frame_id` 確認目前 held frame。

Video/audio 的 `timestamp_ns` 可在同一 process/session 內直接比較。它代表 SDK 交付時間，不代表訊號來源端產生 frame 的時間。

1.16 gvfg_gpu_output_format_t

成員	值	說明
GVFG_GPU_OUTPUT_BGRA8	1	DXGI <code>B8G8R8A8_UNORM</code> byte layout，alpha 為 255
GVFG_GPU_OUTPUT_RGB10A2	2	DXGI <code>R10G10B10A2_UNORM</code> packed <code>uint32_t</code> ，alpha 為 3
GVFG_GPU_OUTPUT_NV12	3	8-bit BT.709 limited-range 4:2:0；Y plane 後接 interleaved UV

1.17 gvfg_gpu_output_buffer_t

由 caller 建立並傳給 `gvfg_gpu_convert_to_buffer()`。

欄位	型別	說明
data	void *	Caller-owned destination buffer，不可為 NULL
data_size	uint64_t	Destination buffer 可用 byte 數
row_bytes	int	Destination stride；NV12 的 Y/UV plane 共用此 stride

欄位	型別	說明
<code>pixel_format</code>	<code>int</code>	要求的 <code>gvfg_gpu_output_format_t</code> 值

例如，要建立一個與輸入 frame 同尺寸的 BGRA8 destination：

```
const int row_bytes = source.width * 4;
const uint64_t buffer_size = (uint64_t)row_bytes * (uint64_t)source.height;
void *pixels = malloc((size_t)buffer_size);

gvfg_gpu_output_buffer_t output = {0};
output.data = pixels;
output.data_size = buffer_size;
output.row_bytes = row_bytes;
output.pixel_format = GVFG_GPU_OUTPUT_BGRA8;

gvfg_status_t status = gvfg_gpu_convert_to_buffer(&source, &output);
/* 使用 pixels 後由 caller 呼叫 free(pixels)。 */
```

Caller 必須確認尺寸乘法沒有 overflow，並在 conversion 返回前保持 destination 有效。

1.18 `gvfg_event_type_t`

成員	值	說明
<code>GVFG_EVENT_UNKNOWN</code>	0	未知事件；正常流程不應依賴此值
<code>GVFG_EVENT_VIDEO_FORMAT_CHANGED</code>	1	輸入 video format 已變更
<code>GVFG_EVENT_VIDEO_INPUT_PLUGIN</code>	2	Video input 已連接
<code>GVFG_EVENT_VIDEO_INPUT_UNPLUG</code>	3	Video input 已中斷

SDK 會建立並管理完整的 channel event 集合；使用者直接透過 `gvfg_poll_channel_event()` 接收。

1.19 `gvfg_handle`

```
typedef struct gvfg_handle_t *gvfg_handle;
```

只能由 `gvfg_create()` 建立、由 `gvfg_destroy()` 銷毀；caller 不可取得、配置或修改其內部內容。

2. GVFG API 函式

說明 `gvfg_capture.h` 的所有公開函式。

函式參數方向以 `[in]`、`[out]` 與 `[in, out]` 標示：`[in]` 由 caller 提供，`[out]` 由函式寫入，`[in, out]` 則同時包含 caller 必須先設定的輸入欄位與函式寫回的結果。

2.1 gvfg_enumerate_devices

```
int gvfg_enumerate_devices(/* [out] */ gvfg_device_info_t *out_devices,  
                           /* [in] */ int max_devices);
```

參數：

- `out_devices`：接收裝置資料的陣列；只查數量時傳 `NULL`。
- `max_devices`：陣列可容納的項目數；超過 `GVFG_MAX_DEVICES` 會被截斷。當 `out_devices == NULL` 時應傳 0。

回傳值：有提供 output array 時為實際寫入數量；只查數量時為可用裝置數；沒有裝置 時為 0。此函式回傳數量，不回傳 `gvfg_status_t`。

2.2 gvfg_create

```
gvfg_status_t gvfg_create(/* [out] */ gvfg_handle *out_handle);
```

- `out_handle`：接收新 handle，不可為 `NULL`。
- 成功：`GVFG_OK`，handle 處於 closed 狀態。
- 失敗：`GVFG_EINVAL`（output pointer 為 `NULL`）。

2.3 gvfg_destroy

```
gvfg_status_t gvfg_destroy(/* [in] */ gvfg_handle handle);
```

- `handle`：`gvfg_create()` 回傳的 handle；可為 `NULL`。
- 回傳：`GVFG_OK`。若仍在 running，SDK 會先停止。返回後 handle 不得再使用。

2.4 gvfg_open_channel

```
gvfg_status_t gvfg_open_channel(/* [in] */ gvfg_handle handle,  
                                /* [in] */ int device_index,  
                                /* [in] */ int channel_index);
```

- `handle`：已建立的 session handle。
- `device_index`：`gvfg_enumerate_devices()` 結果中的 zero-based 陣列位置。
- `channel_index`：`GVFG_CHANNEL_0` 或 `GVFG_CHANNEL_1`。
- 可能回傳：`GVFG_OK`、`GVFG_EINVAL`、`GVFG_ENODEV`、`GVFG_ETIME`、`GVFG_EBUSY`、`GVFG_EIO`。

對同一 handle 可分別 open CH0、CH1。已開啟任一 channel 後，不可使用同一 handle 切換至其他 device index。

2.5 gvfg_get_channel_sdi_info

```
gvfg_status_t gvfg_get_channel_sdi_info(
    /* [in] */ gvfg_handle handle,
    /* [in] */ int channel_index,
    /* [out] */ gvfg_sdi_info_t *out_info);
```

- **handle** : 已 open 指定 channel 的 session。
- **channel_index** : 要查詢的 **GVFG_CHANNEL_0** 或 **GVFG_CHANNEL_1**。
- **out_info** : 接收 SDI 詳細資訊，不可為 NULL。
- **GVFG_OK** : 查詢成功。
- **GVFG_EINVAL** : handle 或 output pointer 為 NULL，或 channel index 無效。
- **GVFG_ESTATE** : 指定 channel 尚未 open。
- 裝置查詢失敗時可能回傳 **GVFG_EIO**。

2.6 Zero-copy mode selection

```
gvfg_status_t gvfg_set_channel_zero_copy_enabled(/* [in] */ gvfg_handle handle,
                                                  /* [in] */ int channel,
                                                  /* [in] */ int enabled);
gvfg_status_t gvfg_get_channel_zero_copy_enabled(/* [in] */ gvfg_handle handle,
                                                  /* [in] */ int channel,
                                                  /* [out] */ int *out_enabled);
```

- **set** 只能在 **gvfg_create()** 後、指定 channel open 前呼叫。
- **enabled** 只接受 0 (copy) 或 1 (zero-copy) ; 每個 channel 預設為 0。
- 指定 channel 已 open 時呼叫 **set** 會回傳 **GVFG_ESTATE** ; 不影響另一條 channel。
- **get** 可查詢指定 channel 的模式 ; **out_enabled** 不可為 NULL。
- Zero-copy mode 的 **frame.data** 是借用的 pointer ; 仍必須以相同 descriptor 呼叫 **gvfg_release_channel_frame()**，且每個 channel 同時最多持有一張 frame。

2.7 gvfg_set_channel_video_format

```
gvfg_status_t gvfg_set_channel_video_format(/* [in] */ gvfg_handle handle,
                                             /* [in] */ int channel_index,
                                             /* [in] */ gvfg_pixel_format_t format);
```

- 支援 **GVFG_PIXFMT_YUY2** 與 **GVFG_PIXFMT_Y210**。
- Channel 必須已 open，且 capture 必須尚未 start 或已 stop。
- 同一 handle 同時只能有一個 channel 選擇 Y210；衝突的要求回傳 **GVFG_EBUSY**。
- YUY2 對應 8-bit color depth，Y210 對應 10-bit。

2.8 Audio capture selection and frame ownership

```
gvfg_status_t gvfg_set_channel_audio_enabled(/* [in] */ gvfg_handle handle,
                                             /* [in] */ int channel_index,
                                             /* [in] */ int enabled);
gvfg_status_t gvfg_get_channel_audio_format(/* [in] */ gvfg_handle handle,
                                             /* [in] */ int channel_index,
                                             /* [out] */ gvfg_audio_format_t *out_format);
gvfg_status_t gvfg_read_channel_audio_frame(/* [in] */ gvfg_handle handle,
                                             /* [in] */ int channel_index,
                                             /* [out] */ gvfg_audio_frame_t *out_frame,
                                             /* [in] */ uint32_t timeout_ms);
gvfg_status_t gvfg_release_channel_audio_frame(/* [in] */ gvfg_handle handle,
                                                /* [in] */ int channel_index,
                                                /* [in] */ const gvfg_audio_frame_t *frame);
```

- 預設只擷取 video。CH0 可在 start 前用 `gvfg_set_channel_audio_enabled()` 啟用 audio。
- `gvfg_start_channel()` 會依 audio enabled 狀態開始 video-only 或 video + audio capture。
- `gvfg_read_channel_audio_frame()` 取得下一個 PCM frame 並交付借用的 descriptor。使用完成後必須呼叫 `gvfg_release_channel_audio_frame()`。
- 未 release 前再次 read 會回傳 `GVFG_ESTATE`。
- `out_frame` 為 NULL 或 release token 被修改時回傳 `GVFG_EINVAL`。
- `timeout_ms` 遵循其他 read API：`0` 不等待，`GVFG_TIMEOUT_INFINITE` 無限等待，超時回傳 `GVFG_ETIMEOUT`。
- Video 與 audio 應由不同 worker thread read；同一 channel 不可同時執行兩個 audio read。
- Stop/close 後 descriptor 立即失效；要跨越 release/stop 保存 PCM 必須先複製。

2.9 `gvfg_start_channel`

```
gvfg_status_t gvfg_start_channel(/* [in] */ gvfg_handle handle,
                                 /* [in] */ int channel_index);
```

- `handle`：已成功 open 的 session handle。
- `GVFG_OK`：開始擷取，或 channel 已經 running。
- `GVFG_EINVAL`：handle 為 NULL。
- `GVFG_ESTATE`：尚未 open 裝置。
- `GVFG_ETIMEOUT`：本次 Start 的即時 signal query 顯示沒有 lock；channel 不會進入 running。
- `GVFG_EBUSY`：裝置或 capture resource 正在使用中。
- `GVFG_EIO`：capture 啟動或 I/O 失敗。

Channel 從 stopped 開始新的 running session 時會執行一次新的 signal query，不沿用 停止前的 signal cache。對已 running 的 channel 再次呼叫 Start 會直接回傳 `GVFG_OK`，不會重新查詢訊號。無訊號時 Application 應等待輸入恢復後再次呼叫 Start。

2.10 gvfg_read_channel_frame

```
gvfg_status_t gvfg_read_channel_frame(/* [in] */ gvfg_handle handle,
                                      /* [in] */ int channel_index,
                                      /* [out] */ gvfg_frame_t *out_frame,
                                      /* [in] */ uint32_t timeout_ms);
```

- **handle** : running session。
- **out_frame** : 接收 frame descriptor，不可為 NULL；呼叫前建議清零。
- **timeout_ms** : 0 為 non-blocking；**GVFG_TIMEOUT_INFINITE** 為無限等待；其他值為毫秒。
- **GVFG_OK** : 成功取得 frame，caller 現在持有一個必須 release 的 token。
- **GVFG_EINVAL** : handle 或 output pointer 為 NULL。
- **GVFG_ESTATE** : 未 running、已有 held frame、已有另一個 read，或等待時被 stop。
- **GVFG_ETIMEOUT** : 期限內沒有 frame。
- **GVFG_EBUSY** 表示 capture resource 正在使用中；**GVFG_EIO** 表示 capture 或 I/O 失敗；**GVFG_ENOTSUP** 表示 SDK 不支援該格式。

Copy mode 與 zero-copy mode 都會回傳借用的 frame buffer。兩者的 pointer 都只保證 有效到對應的 **gvfg_release_channel_frame()**。

2.11 gvfg_release_channel_frame

```
gvfg_status_t gvfg_release_channel_frame(/* [in] */ gvfg_handle handle,
                                         /* [in] */ int channel_index,
                                         /* [in] */ const gvfg_frame_t *frame);
```

- **handle** : 取得該 frame 的同一個 handle。
- **frame** : **gvfg_read_channel_frame()** 原封不動回傳的完整 descriptor。
- **GVFG_OK** : 成功歸還 frame。
- 即使輸入訊號在持有 frame 期間中斷，application 仍應對原本成功取得的 frame 呼叫一次 release。
- **GVFG_EINVAL** : NULL 或 token 內容與 held frame 不符。
- **GVFG_ESTATE** : channel 未開啟，或目前沒有可 release 的 held frame。

2.12 gvfg_gpu_convert_to_buffer

```
gvfg_status_t gvfg_gpu_convert_to_buffer(
    /* [in] */ const gvfg_frame_t *source,
    /* [in] */ const gvfg_gpu_output_buffer_t *output);
```

- **source** : 有效的 YUY2/Y210 frame；若來自 read，必須尚未 release。
- **output** : caller 填好的 destination descriptor。
- **GVFG_OK** : 同步轉換及 copy 完成。
- **GVFG_EINVAL** : NULL、尺寸、stride、buffer size、奇偶尺寸或 overflow 錯誤。
- **GVFG_ENOTSUP** : input/output format 不支援。
- **GVFG_EIO** : D3D/GPU resource、dispatch 或 readback 失敗。

2.13 gvfg_gpu_convert_to_bgra8

```
gvfg_status_t gvfg_gpu_convert_to_bgra8(  
    /* [in] */ const gvfg_frame_t *source,  
    /* [out] */ void *destination,  
    /* [in] */ uint64_t destination_size,  
    /* [in] */ int row_bytes);
```

- **source** : 有效且尚未 release 的 YUY2/Y210 frame。
- **destination** : caller-owned BGRA8 buffer。
- **destination_size** : buffer byte 數，至少 **row_bytes * source->height**。
- **row_bytes** : destination stride，至少 **source->width * 4**。
- 回傳狀態與通用 GPU conversion 相同。

2.14 gvfg_gpu_convert_to_rgb10a2

```
gvfg_status_t gvfg_gpu_convert_to_rgb10a2(  
    /* [in] */ const gvfg_frame_t *source,  
    /* [out] */ void *destination,  
    /* [in] */ uint64_t destination_size,  
    /* [in] */ int row_bytes);
```

參數與 BGRA8 wrapper 相同，但 destination layout 是 packed RGB10A2；每列仍至少 **source->width * 4** bytes。回傳狀態與通用 GPU conversion 相同。

2.15 gvfg_gpu_convert_to_nv12

```
gvfg_status_t gvfg_gpu_convert_to_nv12(  
    /* [in] */ const gvfg_frame_t *source,  
    /* [out] */ void *destination,  
    /* [in] */ uint64_t destination_size,  
    /* [in] */ int row_bytes);
```

- **source** : 有效且尚未 release 的 YUY2/Y210 frame；width、height 必須為偶數。
- **destination** : caller-owned NV12 buffer。
- **destination_size** : 至少 **row_bytes * (height + height / 2)**。
- **row_bytes** : Y 與 UV plane 共用的 stride，至少為 **width**。
- 回傳狀態與通用 GPU conversion 相同。

2.16 gvfg_poll_channel_event

```
gvfg_status_t gvfg_poll_channel_event(/* [in] */ gvfg_handle handle,  
    /* [in] */ int channel_index,  
    /* [out] */ gvfg_event_type_t *out_event_type,  
    /* [in] */ uint32_t timeout_ms);
```

- `handle` : running session。
- `out_event_type` : 接收一個 `gvfg_event_type_t` 事件種類，不可為 NULL。
- `timeout_ms` : 規則與 `read` 相同。
- `GVFG_OK` : 成功取出一個事件。
- `GVFG_EINVAL` : `handle` 或 output pointer 為 NULL。
- `GVFG_ESTATE` : 未 running，或等待時被 stop。
- `GVFG_ETIMEOUT` : 期限內沒有事件。

2.17 `gvfg_stop`

```
gvfg_status_t gvfg_stop(/* [in] */ gvfg_handle handle);
```

- `handle` : session handle，不可為 NULL。
- `GVFG_OK` : 成功；已經 stopped 也視為成功。
- `GVFG_EINVAL` : `handle` 為 NULL。

此函式會停止 capture、喚醒等待中的 read/poll，並使未 release frame 失效。

2.18 `gvfg_stop_channel`

```
gvfg_status_t gvfg_stop_channel(/* [in] */ gvfg_handle handle,  
                                /* [in] */ int channel_index);
```

- 只停止指定 channel；`gvfg_stop()` 會停止同一 handle 已開啟的所有 channel。
- 尚未 open 的 channel 回傳 `GVFG_ESTATE`。

2.19 `gvfg_get_channel_signal_status`

```
gvfg_status_t gvfg_get_channel_signal_status(  
    /* [in] */ gvfg_handle handle,  
    /* [in] */ int channel_index,  
    /* [out] */ gvfg_signal_status_t *out_status);
```

- `handle` : 已 open 的 session。
- `out_status` : 接收 signal status，不可為 NULL。
- `GVFG_OK` : 查詢成功；沒有訊號時仍成功，但 `connected == 0`。
- `GVFG_EINVAL` : `handle` 或 output pointer 為 NULL。
- `GVFG_ESTATE` : 尚未 open 裝置。
- Capture I/O 失敗時可能回傳 `GVFG_EIO`。

Channel 尚未 running 時，此 API 會更新目前的 signal status；running 期間回傳最近一次觀察到的狀態。Format changed、input plug-in 與 input unplug event 發生後，後續查詢會反映更新後的狀態。

2.20 `gvfg_get_channel_runtime_info`

```
gvfg_status_t gvfg_get_channel_runtime_info(  
    /* [in] */ gvfg_handle handle,  
    /* [in] */ int channel_index,  
    /* [out] */ gvfg_runtime_info_t *out_info);
```

- `handle` : 已 open 或 running 的 session。
- `out_info` : 接收 runtime statistics，不可為 NULL。
- `GVFG_OK` : 查詢成功。
- `GVFG_EINVAL` : handle 或 output pointer 為 NULL。
- `GVFG_ESTATE` : 指定 channel 尚未 open。

2.21 `gvfg_get_version`

```
const char *gvfg_get_version(void);
```

- 回傳目前實際載入的 GVFG runtime DLL 版本，例如 `"1.0.1"`。
- 回傳值是靜態 null-terminated 字串，caller 不可 free。
- 可用於 log、問題回報，以及確認 header、DLL 是否來自同一版本。

2.22 `gvfg_pixel_format_name`

```
const char *gvfg_pixel_format_name(/* [in] */ int pixel_format);
```

- `pixel_format` : `gvfg_pixel_format_t` 或其他整數值。
- 回傳：靜態英文字串 `"YUY2"`、`"Y210"` 或 `"UNKNOWN"`。Caller 不可 free。

2.23 `gvfg_strerror`

```
const char *gvfg_strerror(/* [in] */ gvfg_status_t status);
```

- `status` : 任一 GVFG status code。
- 回傳：靜態、null-terminated 英文說明字串；未知值回傳 unknown 類型說明。Caller 不可 free。

2.24 `gvfg_get_channel_last_sdk_error_detail`

```
gvfg_status_t gvfg_get_channel_last_sdk_error_detail(/* [in] */ gvfg_handle handle,  
    /* [in] */ int channel_index,  
    /* [out] */ char *out_message,  
    /* [in] */ uint32_t out_message_size);
```

此函式提供公開 `GVFG_E*` 失敗狀態的補充說明。回傳內容只描述應用程式可採取行動的 公開錯誤資訊。

- 複製指定 channel 最近一次 fault 或被拒絕操作的 UTF-8 詳細說明；即使該 channel open 失敗仍可查詢。

- `gvfg_read_channel_frame()` / `gvfg_poll_channel_event()` 的 timeout、non-blocking 無資料，以及正常 stop 喚醒 waiter 都不會覆寫此內容。
- 每次呼叫 `gvfg_start_channel()` 時會清除先前保存的錯誤說明。
- 同一 channel 若被多執行緒同時操作，內容可能被後續錯誤覆蓋；應在失敗後立即取得。
- `GVFG_EINVAL`：handle/message 為 NULL、channel index 無效，或 buffer size 為 0。

3. Preview API 完整參考

本節說明選用的 `gvfg_preview.h`。Preview status 與 core `gvfg_status_t` 是不同 enum，不可混用。

3.1 Preview 型別

`gvfg_preview_status_t`：

成員	說明
<code>GVFG_PREVIEW_OK</code>	成功
<code>GVFG_PREVIEW_EINVAL</code>	NULL、尺寸或參數錯誤
<code>GVFG_PREVIEW_ESTATE</code>	尚未建立 / attach，或狀態不允許操作
<code>GVFG_PREVIEW_ENOTSUP</code>	不支援 frame 格式
<code>GVFG_PREVIEW_ERENDER</code>	D3D/DXGI render 或 Present 失敗

`gvfg_preview_handle` 是 opaque pointer，只能由 preview create/destroy 管理。

`gvfg_preview_pixel_format_t`：

成員	值	說明
<code>GVFG_PREVIEW_PIXFMT_YUY2</code>	1	Packed 8-bit YUV 4:2:2；值與 core YUY2 相同
<code>GVFG_PREVIEW_PIXFMT_Y210</code>	2	Packed 10-bit YUV 4:2:2；值與 core Y210 相同
<code>GVFG_PREVIEW_PIXFMT_GRAY16</code>	4	每 pixel 一個 16-bit unsigned grayscale component
<code>GVFG_PREVIEW_PIXFMT_RGBA16</code>	5	每 pixel 四個 16-bit UNORM component：R、G、B、A

`gvfg_preview_frame_t`：

欄位	說明
<code>data</code> 、 <code>data_size</code>	Caller-owned frame pointer 與 byte 數
<code>width</code> 、 <code>height</code>	Frame pixel 尺寸
<code>pixel_format</code>	<code>gvfg_preview_pixel_format_t</code>
<code>bit_depth</code>	Frame bit depth
<code>row_bytes</code>	Source row stride，通常對應 <code>gvfg_frame_t.row_stride_bytes</code>
<code>frame_id</code>	Frame 識別值，通常對應 <code>gvfg_frame_t.frame_id</code>

gvfg_preview_info_t :

欄位	說明
active	非 0 表示 preview pipeline 已啟用
width、height、bit_depth	最近使用的 frame 資訊
pixel_format[32]	Null-terminated 格式名稱
adapter_name[160]	Null-terminated D3D adapter 名稱
adapter_index	使用的 adapter 索引

gvfg_preview_stats_t :

欄位	說明
present_fps	最近五秒成功 Present 的速率
presented_frames	成功 Present 的累積 frame 數
skipped_presents	Non-blocking swapchain busy 而略過的累積次數

gvfg_preview_delivery_stats_t : 以下 counter 從 create 或前一次 shutdown 開始累積；configure 與 clear 不會重設。presented 表示 DXGI 接受 Present，不代表已實際 scanout。

欄位	說明
submitted	Preview 接收到並納入 delivery 統計的 render request 數
presented	已被 DXGI Present 接受的 frame 數
replaced	在 50 ms age limit 後被捨棄的 queued frame 數
busy	遇到 upload slot busy 或 Present busy 的 request 數
failed	已提交但後續處理失敗的 frame 數
cancelled	Shutdown 等流程取消的 frame 數
in_flight	目前仍在 pipeline 中的 frame 數
last_presented_id	最近成功 Present 的 frame_id
present_busy	因 Present busy 而略過的累積次數
slots_busy	因內部 upload slot busy 而略過的累積次數
last_busy_id	最近一次 busy frame 的 frame_id

正常情況下：

```
submitted = presented + replaced + busy + failed + cancelled + in_flight
```


3.2 Preview 函式

函式	參數與行為
<code>gvfg_preview_get_delivery_stats(handle, out_stats)</code>	[in] <code>handle</code> ; [out] <code>out_stats</code> 。取得 frame delivery lifetime counters ; output 不可為 NULL
<code>gvfg_preview_wait_idle(handle, timeout_ms)</code>	[in] <code>handle</code> , <code>timeout_ms</code> 。停止提交新 frame 後等待 queued work 完成 ; 仍有工作時回傳 <code>GVFG_PREVIEW_ESTATE</code>
<code>gvfg_preview_create(out_handle)</code>	[out] <code>out_handle</code> 。接收新 preview handle ; 不可為 NULL
<code>gvfg_preview_destroy(handle)</code>	[in] <code>handle</code> 。銷毀 handle 與相關資源 ; 返回後不得再使用
<code>gvfg_preview_attach_window(handle, native_window_handle)</code>	[in] <code>handle</code> , <code>native_window_handle</code> 。將 preview 接到 Windows <code>HWND</code> ; 兩參數都必須有效
<code>gvfg_preview_prepare(handle, width, height, bit_depth)</code>	[in] <code>handle</code> , <code>width</code> , <code>height</code> , <code>bit_depth</code> 。在持有第一張 capture frame 前預先建立 GPU resources
<code>gvfg_preview_render_frame(handle, frame)</code>	[in] <code>handle</code> , <code>frame</code> 。返回前同步 upload caller data ; GPU conversion 與 Present 由內部 worker 繼續執行
<code>gvfg_preview_clear(handle)</code>	[in] <code>handle</code> 。將最後呈現的畫面替換成黑畫面
<code>gvfg_preview_get_info(handle, out_info)</code>	[in] <code>handle</code> ; [out] <code>out_info</code> 。寫入目前 pipeline/frame/adaptor 資訊
<code>gvfg_preview_get_stats(handle, out_stats)</code>	[in] <code>handle</code> ; [out] <code>out_stats</code> 。寫入 Present 統計
<code>gvfg_preview_shutdown(handle)</code>	[in] <code>handle</code> 。關閉目前 preview pipeline , 但保留 handle 供後續使用或 destroy
<code>gvfg_preview_strerror(status)</code>	[in] <code>status</code> 。回傳靜態英文錯誤字串 ; caller 不可 free

除 `gvfg_preview_strerror()` 外，preview 函式都回傳 `gvfg_preview_status_t` 。實際錯誤 應以函式回傳值判斷，並使用 `gvfg_preview_strerror()` 記錄說明。

`gvfg_preview_render_frame()` 返回後，caller 即可 release 或重用原始 frame memory ; 函式返回不代表該 frame 已完成 GPU conversion、Present 或實際顯示。需要在停止提交後確認 內部工作已完成時，呼叫 `gvfg_preview_wait_idle()`。

4. Audio Playback Helper 完整參考

本節說明選用的 `gvfg_audio_playback.h` 。`gvfg_audio_playback.dll` 是獨立的 Windows WASAPI helper，不依賴 `gvfg.dll` 。Application 仍負責從 Capture API 取得 PCM、複製資料並 release SDK audio frame，再把 application-owned PCM 送入 helper。

4.1 Audio Playback 型別

`gvfg_audio_playback_status_t` :

成員	說明
<code>GVFG_AUDIO_PLAYBACK_OK</code>	成功

成員	說明
GVFG_AUDIO_PLAYBACK_EINVAL	NULL、空資料、格式、frame alignment 或音量參數錯誤
GVFG_AUDIO_PLAYBACK_ESTATE	Player 尚未啟動或目前狀態不允許操作
GVFG_AUDIO_PLAYBACK_ENODEV	找不到 Windows 預設 audio output
GVFG_AUDIO_PLAYBACK_EFORMAT	WASAPI 不支援指定 PCM 格式
GVFG_AUDIO_PLAYBACK_EQUEUE_FULL	內部 bounded queue 已滿；本次 packet 未接受
GVFG_AUDIO_PLAYBACK_EIO	COM、WASAPI 或 playback worker 操作失敗

gvfg_audio_player 是 opaque pointer，只能由 audio player create/destroy 管理。

gvfg_audio_playback_format_t：

欄位	說明
sample_rate	每秒 sample 數
channels	Interleaved PCM channel 數
bits_per_sample	每個 PCM sample 的 bit 數

4.2 Audio Playback 函式

函式	參數與行為
gvfg_audio_player_create(out_player)	[out] out_player。建立 player；不可為 NULL
gvfg_audio_player_start(player, format)	在 Windows 預設輸出裝置啟動 shared-mode WASAPI playback
gvfg_audio_player_write(player, data, data_size)	複製完整 interleaved PCM frame 到 bounded queue；返回後 caller 可立即重用資料
gvfg_audio_player_write_timed(player, data, data_size, audio_timestamp_ns)	連同 SDK audio delivery timestamp 排入 queue，供 helper 與 video timestamp 做有限度同步
gvfg_audio_player_update_video_timestamp(player, video_timestamp_ns)	提供同一 SDK monotonic clock domain 的最新 video delivery timestamp
gvfg_audio_player_reset_timeline(player)	清除 queued PCM、重設 WASAPI stream，並建立新的 A/V sync timeline
gvfg_audio_player_set_volume(player, volume)	設定 PCM16 software gain；範圍 0.0 到 2.0
gvfg_audio_player_stop(player)	停止 worker 並清除 queued PCM；可重複呼叫
gvfg_audio_player_destroy(player)	停止並銷毀 player；NULL 可安全忽略
gvfg_audio_player_strerror(status)	回傳靜態英文錯誤字串；caller 不可 free

Helper 的 queue 上限約為 500 ms PCM。GVFG_AUDIO_PLAYBACK_EQUEUE_FULL 表示本次 packet 未被接受；capture thread 不應為等待播放而延後 release SDK frame。write_timed() 使用的是 SDK delivery timestamp，不是來源端硬體 PTS，因此同步只適合限制明顯的短期 lead/lag，不能作為 端到端硬體 A/V sync 的量測依據。